Initial System Design (Small Deployment)

1. Overview

We start with a **small-scale deployment** of an application (Beigel) designed for:

- Reading posts about nearby cafes
- Creating posts

At this stage:

- **Low traffic**
- **Simple architecture**
- Focus on **fast development & simplicity**

2. System Components

Client-Side (Consumer Apps)

1. Browser App

- Built using **ReactJS**
- Communicates via HTTP APIs
- Uses **Brotli compression** (better than Gzip)

Why Brotli?

- Smaller bundle size → faster load time
- Better performance for users

2. Mobile Apps

- iOS App
- Android App

Communicate directly with backend via APIs (JSON over HTTP)

3. Server-Side Components

Frontend Service (BFF – Backend for Frontend)

- Acts as **middle layer for browser**
- Typically implemented in Node.js

Responsibilities:

- API aggregation
- Authentication handling
- Response formatting

Backend Service

- Stateless service (Go / Java)
- Handles:
    - Business logic
    - Data processing
    - API endpoints

Stateless = can scale easily later

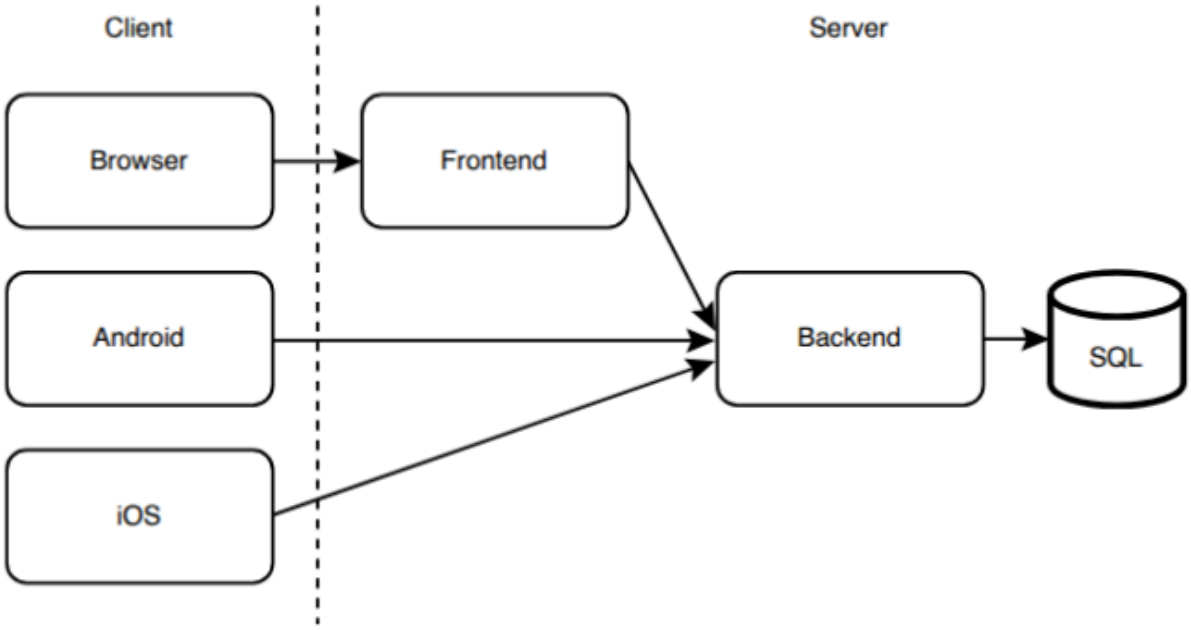
Database

- Single **SQL database**
- Hosted on one cloud machine

Used for:

- Users
- Posts
- Locations

4. High-Level Architecture



5. Request Flow

Browser Flow:

1. Browser → Frontend Service
2. Frontend → Backend Service
3. Backend → Database
4. Response returns back same path

Mobile Flow:

1. Mobile App → Backend Service
2. Backend → Database
3. Response → Mobile

6. DNS Routing

- DNS maps domain → IP address
- All traffic routed to **single host(s)**

Example:

- Browser → Frontend host
- Mobile → Backend host

7. Deployment Strategy (Initial Stage)

- Each service runs on **one cloud instance**
- Hosted in **same data center**

Why?

- Simplicity
- Low cost
- Easy debugging

8. Important Design Decisions

Why Separate Frontend & Backend?

- Reduce load on backend
- Customize responses for UI
- Improve performance

Why Stateless Backend?

- Easy horizontal scaling later
- No session dependency

Why Single DB Initially?

- Simplicity
- No need for sharding/replication yet

9. Limitations of This Setup

1. Single Point of Failure

- If backend fails → entire system down

2. No Scalability

- One instance per service

3. Database Bottleneck

- All reads/writes go to one DB

4. No Load Balancing

- Cannot handle traffic spikes

10. How This System Will Evolve

This is **VERY IMPORTANT for interviews**

Step 1:

Add Load Balancer

Step 2:

Multiple Backend Instances

Step 3:

Add Cache (Redis/CDN)

Step 4:

Database Replication

Step 5:

Sharding + Microservices

11. Final Summary

- Initial system = simple + low traffic
- Clients: Web + Mobile
- Services: Frontend + Backend
- DB: Single SQL instance
- Stateless backend → future scaling
- Major issue: **no scalability & SPOF**